Confluent Cloud for Apache Flink — Reference Guide

> A single-source reference for the Confluent Cloud (CC) for Apache Flink architecture, resources, deployment, and operational details in this repository.

Table of Contents

- 1.0 Environment & Infrastructure
  - 1.1 Cloud provider and region
  - 1.2 Confluent environment
  - 1.3 Kafka cluster
  - 1.4 Schema Registry
  - 1.5 Flink compute pool
- 2.0 Identity & Access
  - 2.1 Service accounts
  - 2.2 Role bindings
  - 2.3 API key rotation
- 3.0 Flink SQL Statements & Artifacts
  - 3.1 Statement execution order
  - 3.2 UDF artifact
  - 3.3 Streaming enrichment job
- 4.0 UDF Implementation
  - 4.1 Class and state model
  - 4.2 Build toolchain
- 5.0 Terraform Configuration
  - 5.1 Provider and backend
  - 5.2 Variables
  - 5.3 Dependency graph
- 6.0 Deployment
  - 6.1 Deploy to Confluent Cloud
  - 6.2 Tear down
- 7.0 CC vs CP Deployment Comparison
- 8.0 Performance Specifications
  - 8.1 Scaling model
  - 8.2 Limits
  - 8.3 Latency and delivery guarantees
- 9.0 Key File Locations
- 10.0 Resources

1.0 Environment & Infrastructure

1.1 Cloud provider and region

Setting	Value

Setting	Value
Cloud	AWS
Region	us-east-1

1.2 Confluent environment

Setting	Value
Resource type	`confluent_environment`
Name	`ptf-udf`
Stream Governance	ESSENTIALS package

1.3 Kafka cluster

Setting	Value
Resource type	`confluent_kafka_cluster`
Name	`ptf-udf`
Availability	SINGLE_ZONE
Cluster type	STANDARD
Cloud / Region	AWS us-east-1
Topics	`user_events`, `enriched_events`

1.4 Schema Registry

Auto-provisioned with the ESSENTIALS Stream Governance package. Referenced via:

```
data.confluent_schema_registry_cluster.ptf_udf
```

1.5 Flink compute pool

Setting	Value
Resource type	`confluent_flink_compute_pool`
Name	`apache_flink_flink_statement_runner`
Max CFU	10 (default — increase to 50 for production)
Cloud / Region	AWS us-east-1

2.0 Identity & Access

2.1 Service accounts

Resource	Name	Purpose
`confluent_service_account.flink_sql_runner`	`ptf-udf`	Executes Flink SQL statements

2.2 Role bindings

The `flink_sql_runner` service account has the following role bindings:

Role	Scope
`FlinkDeveloper`	Organization-wide
`ResourceOwner`	Kafka topics (`topic=*`)
`ResourceOwner`	Schema Registry subjects (`subject=*`)
`ResourceOwner`	Transactional IDs (`transactional-id=*`)
`Assigner`	API key rotation module

2.3 API key rotation

Setting	Value
Module source	`github.com/j3-signalroom/iac-confluent-api_key_rotation-tf_module`
Rotation interval	30 days (configurable via `day_count`)
Keys retained	2 minimum (configurable via `number_of_api_keys_to_retain`)

Flink statements authenticate using:

```
credentials {
  key    = module.flink_api_key_rotation.active_api_key.id
  secret = module.flink_api_key_rotation.active_api_key.secret
}
```

3.0 Flink SQL Statements & Artifacts

3.1 Statement execution order

All statements are Terraform-managed `confluent_flink_statement` resources with explicit `depends_on` ordering:

Step	Resource	SQL

Step	Resource	SQL
1	`drop_user_events`	`DROP TABLE IF EXISTS user_events;`
2	`user_events_source`	`CREATE TABLE user_events (user_id STRING, event_type STRING, payload STRING);`
3	`insert_user_events`	`INSERT INTO user_events VALUES (...)` — 6 sample rows
4	`drop_enriched_events`	`DROP TABLE IF EXISTS enriched_events;`
5	`enriched_events_sink`	`CREATE TABLE enriched_events (user_id STRING, event_type STRING, payload STRING, session_id BIGINT, event_count BIGINT, last_event STRING);`
6	`create_udf`	`CREATE FUNCTION IF NOT EXISTS user_event_enricher AS 'ptf.UserEventEnricher' USING JAR 'confluent-artifact://<artifact-id>';`
7	`insert_enriched_events`	`INSERT INTO enriched_events SELECT ... FROM TABLE(user_event_enricher(...));` — **long-running streaming job**

3.2 UDF artifact

Setting	Value
Resource type	`confluent_flink_artifact`
Display name	`ptf-udf`
Content format	JAR
Source	`../java/app/build/libs/app-1.0.0-SNAPSHOT.jar`
URI format	`confluent-artifact://<artifact-id>`

3.3 Streaming enrichment job

Step 7 is a continuously running streaming job that reads from `user_events`, partitions by `user_id`, applies the `user_event_enricher` PTF UDF, and writes enriched output to `enriched_events`.

4.0 UDF Implementation

4.1 Class and state model

Detail	Value
Package	`ptf`
Class	`UserEventEnricher extends ProcessTableFunction<Row>`

Detail	Value
Source	`examples/ptf_udf/java/app/src/main/java/ptf/UserEventEnricher.java`

State (`UserEventEnricher.UserState`):

Field	Type	Description
`eventCount`	`long`	Running count within session
`sessionId`	`long`	Monotonically increasing session ID (incremented on `login`)
`lastEvent`	`String`	Most recent event type

Processing logic:

- Input: Table of `user_events` partitioned by `user_id`
- A `login` event starts a new session (increments `sessionId`, resets `eventCount`)
- Each row is immediately emitted with enriched fields via `collect(Row.of(...))`

4.2 Build toolchain

Setting	Value
Build tool	Gradle (Kotlin DSL)
Java version	17
Flink version	2.1.0
Kafka connector	4.0.1-2.0
Shadow JAR plugin	9.0.0-beta12
Output JAR	`app-1.0.0-SNAPSHOT.jar`

5.0 Terraform Configuration

5.1 Provider and backend

Setting	Value
Terraform backend	Terraform Cloud
Organization	`signalroom`
Workspace	`apache-flink-kickstarter-ii-ptf-udf`
Confluent provider	`confluentinc/confluent` v2.65.0

5.2 Variables

Variable	Description	Default

Variable	Description	Default
`confluent_api_key`	Confluent Cloud API key	—
`confluent_api_secret`	Confluent Cloud API secret	—
`day_count`	API key rotation interval (days)	30
`number_of_api_keys_to_retain`	Minimum keys to retain	2

5.3 Dependency graph

```
Environment
 ├── Kafka Cluster
 │    └── Schema Registry (auto-provisioned)
 ├── Service Account + Role Bindings
 │    └── API Key Rotation Module
 ├── Flink Compute Pool
 └── Flink Statements (ordered via depends_on)
      ├── DROP / CREATE user_events
      ├── INSERT sample data
      ├── DROP / CREATE enriched_events
      ├── Upload Artifact (UDF JAR)
      ├── CREATE FUNCTION
      └── INSERT INTO enriched_events (streaming job)
```

6.0 Deployment

6.1 Deploy to Confluent Cloud

```
make deploy-cc-ptf-udf \
  CONFLUENT_API_KEY=<key> \
  CONFLUENT_API_SECRET=<secret>
```

This target:

1. Builds the UDF fat JAR via `./gradlew clean shadowJar`
2. Runs `terraform init` and `terraform apply -auto-approve`
3. Generates a Terraform visualization PNG

6.2 Tear down

```
make teardown-cc-ptf-udf \
  CONFLUENT_API_KEY=<key> \
  CONFLUENT_API_SECRET=<secret>
```

Runs `terraform destroy -auto-approve` to remove all Confluent Cloud resources.

7.0 CC vs CP Deployment Comparison

Aspect	Confluent Cloud	Confluent Platform (Minikube)
Infrastructure	Confluent-managed	Minikube + K8s
SQL submission	Terraform `confluent_flink_statement`	`sql-client.sh` on JobManager pod
JAR delivery	`confluent_flink_artifact` → artifact store	`kubectl exec` → pod filesystem
State management	Terraform state	Flink session cluster
Compute sizing	CFU-based (max 50 per statement)	K8s pod resources (CPU/memory)
Authentication	Cloud API key / secret	N/A (pod-local)
Entry point	`make deploy-cc-ptf-udf`	`make deploy-cp-ptf-udf`
Monitoring	Confluent Cloud Console	Flink UI (`make flink-ui`)
Same UDF code?	Yes	Yes

8.0 Performance Specifications

8.1 Scaling model

- **Flink SQL Autopilot** automatically adjusts parallelism based on consumer lag ("Messages Behind" metric).
- Pools scale down to **zero** when idle — no charges for idle pools.
- **Pricing**: ~$0.21 per CFU-hour. Minimum 1 CFU-minute per statement.
- Up to **4 UDF instances** consolidate into a single CFU.

8.2 Limits

Limit	Value
Max CFUs per pool (standard)	50
Max CFUs per pool (Limited Availability)	1,000
Max CFUs per statement	50
Query text size	4 MB
State size soft limit per statement	500 GB
State size hard limit per statement	1,000 GB
Target state per task manager	~10 GB

8.3 Latency and delivery guarantees

Guarantee	Latency	Requirement
Exactly-once	~1 minute	`isolation.level: read_committed`
At-least-once	Sub-100ms	`isolation.level: read_uncommitted` (possible duplicates)

See ccaf-performance_specs-high_throughput_workloads.md for full performance best practices.

9.0 Key File Locations

Component	Path
CC Terraform configs	`examples/ptf_udf/cc_deploy/`
CC deployment script	`scripts/deploy-cc-ptf-udf.sh`
CP deployment script	`scripts/deploy-cp-ptf-udf.sh`
UDF source code	`examples/ptf_udf/java/app/src/main/java/ptf/UserEventEnricher.java`
Gradle build	`examples/ptf_udf/java/app/build.gradle.kts`
Flink K8s manifest	`k8s/base/flink-basic-deployment.yaml`
CP K8s manifest	`k8s/base/confluent-platform-c3++.yaml`
Makefile	`Makefile`
Performance specs	`docs/ccaf-performance_specs-high_throughput_workloads.md`

10.0 Resources

- Confluent Cloud for Apache Flink Overview
- Flink Compute Pools
- Flink SQL Autopilot
- Flink Billing (CFUs)
- Delivery Guarantees & Latency
- Create a User-Defined Function
- Flink SQL Query Profiler

- Private Networking for Flink
- Confluent Terraform Provider